SRI SHAKTHI
INSTITUTE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution
Coimbatore - 62.
POWERING THE YOUTH
EMPOWERING THE NATION
NBA NATIONAL BOARD OF ACCREDITATION For CSE,MECH,EEE, IT
ACCREDITED WITH GRADE A NAAC

MERN PROJECT

NUTRI AI APP

DONE BY:
PREETHI KRISHNA B
III – CSE C

ABSTRACT

NUTRI AI is a MERN-stack-based web application that uses Artificial Intelligence to provide personalized diet and fitness guidance based on individual user requirements. The system collects user information such as height, weight, fitness goal, and other relevant inputs to generate an AI-based diet chart designed to support either weight loss or weight gain. It also provides an estimated daily water intake to help users maintain proper hydration.

One of the key features of NUTRI AI is its AI-powered food analysis system. Users can upload a photo of a food item, and the system analyzes the image to identify the food and provide an estimated calorie value. Based on the user's fitness goal and dietary requirements, the application can indicate whether the food is suitable to consume at that time. The system provides an easy-to-understand interface that helps users monitor their food choices and follow their personalized diet plan.

The application is developed using the MERN stack, consisting of MongoDB, Express.js, React.js, and Node.js, with AI services integrated for personalized diet generation and food image analysis. NUTRI AI aims to make diet management more accessible, personalized, and convenient by combining web technologies with AI-based recommendations in a single platform.

TECH STACK

Frontend

Nutri AI's frontend is built using React.js, providing a component-based architecture for creating an interactive and responsive nutrition management interface. React is used to develop different views of the application, including the user profile, personalized diet chart, water-intake tracking, food calorie analysis, and fitness goal sections. The frontend uses reusable components so that features such as diet cards, calorie information, food-upload interfaces, and progress indicators can be developed and maintained independently.

The application provides an interactive interface where users can enter details such as height, weight, and fitness goal. Based on these inputs, the system displays an AI-generated diet plan designed according to the user's selected goal, such as weight loss or weight gain. Users can also upload food images through the frontend to obtain calorie-related information and determine whether the food is suitable according to their current diet plan.

Backend

The backend of Nutri AI is developed using **Node.js and Express.js**, providing the REST API layer between the React frontend and MongoDB database. The backend manages user information, fitness goals, diet-plan requests, food-analysis requests, and other application-related operations.

Node.js provides the runtime environment for the server-side application, while Express.js handles HTTP requests, routing, and API communication. The backend also acts as the communication layer between the frontend and AI services used for generating personalized diet recommendations and analyzing uploaded food images.

Database

Nutri AI uses **MongoDB** as its primary database for storing application data. MongoDB stores user profiles, height and weight information, fitness goals, diet-related data, water-intake information, and other user-specific records.

Mongoose is used as the Object Data Modeling (ODM) layer between Node.js and MongoDB. It provides schemas and models that define the structure of the application's data and simplifies database operations through the Express.js backend.

API Integration

Artificial Intelligence is one of the core components of Nutri AI. The system uses AI to generate personalized diet recommendations based on user-provided information such as height, weight, and whether the user wants to lose or gain weight.

The AI component also supports food image analysis. Users can upload a photograph of a food item, which is analyzed to identify the food and provide an estimated calorie value. The system can then compare the food information with the user's selected fitness goal and provide a suitable recommendation.

- **REST API Integration** – Connects the React frontend with the Node.js and Express.js backend to securely exchange user and nutrition-related data.
- **AI API Integration** – Connects the backend with AI services to generate personalized diet charts based on height, weight, and fitness goals.
- **Food Image Analysis API** – Sends uploaded food images to an AI-based image analysis service to identify food items and estimate their calorie content.

MERN Architecture

Nutri AI follows the MERN stack architecture, consisting of four primary technologies:

- MongoDB – Stores user profiles, fitness information, diet data, and application records.
- Express.js – Provides the REST API and server-side routing.
- React.js – Builds the interactive frontend and user interfaces.
- Node.js – Provides the server-side JavaScript runtime.

The overall flow of the system is:

React Frontend → Express.js API → Node.js Backend → Mongoose → MongoDB

AI services are integrated with the backend to generate personalized diet charts and analyze food images.

This architecture provides a clear separation between the presentation layer, application logic, database layer, and AI functionality, making Nutri AI easier to develop, maintain, and extend.

MODULES

Module 1: User Authentication & Management

This module handles user registration, login, and account management within the Nutri AI application. Users can create an account and log in to access their personalized nutrition information and diet plans. Authentication is handled through the backend, with user information stored securely in MongoDB.
The module provides the foundation for associating nutrition-related activities with individual users. Once authenticated, users can enter their personal information such as height, weight, and fitness goals. The backend manages user-related requests through Express.js routes and Mongoose models, while the React frontend provides the corresponding registration and login interfaces.

Module 2: User Profile & Fitness Goal Management

This module collects and manages the user's basic fitness information required for personalized nutrition recommendations. Users can provide details such as height, weight, and their desired fitness goal, including weight loss or weight gain.
The entered information is stored in MongoDB and can be updated whenever the user's requirements change. The React frontend provides forms and interfaces for entering and modifying the user's fitness information.
The collected profile information acts as the primary input for the AI-based recommendation system. It allows Nutri AI to generate diet recommendations according to the individual user's requirements rather than providing the same diet plan to every user.

Module 3: AI-Based Personalized Diet Generation

The AI-Based Personalized Diet Generation module is one of the core modules of Nutri AI. It generates a personalized diet chart based on the user's height, weight, fitness goal, and other provided information.
The user's information is sent from the React frontend to the backend through REST APIs. The backend communicates with the integrated AI service, which processes the input and generates suitable dietary recommendations.
The generated diet chart is then returned to the frontend and displayed in an easy-to-understand format. Users can view recommended meals and food choices according to their selected goal, such as weight loss or weight gain.
This module reduces the need for users to manually create a diet plan and provides personalized recommendations through AI integration.

Module 4: Food Image & Calorie Analysis

The Food Image & Calorie Analysis module allows users to upload photographs of food items through the Nutri AI application. The uploaded image is processed using an AI-based image analysis service.

The system analyzes the image to identify the food item and provides an estimated calorie value. The analysis result is returned to the application through the backend API and displayed on the React frontend.

This module helps users understand the approximate nutritional value of the food they are planning to consume. It provides a convenient alternative to manually searching for individual food items and their calorie information.

Module 5: Food Recommendation & Diet Compatibility

This module evaluates the analyzed food information in relation to the user's fitness goal and personalized diet plan

The application provides an easy-to-understand indication of whether the food can be included in the user's current diet plan. The recommendation is presented through the frontend along with the available calorie information.

This module connects the AI food analysis functionality with the personalized diet system, allowing users to make more informed food choices according to their selected fitness objective.

Module 6: Water Intake Management

The Water Intake Management module provides users with an estimated daily water intake target based on their personal information and fitness requirements.

The recommended water intake is displayed through the Nutri AI dashboard so that users can easily monitor their daily hydration target. The frontend provides visual indicators or progress information to make the hydration information easier to understand.

Module 7: Nutrition Dashboard & Progress Tracking

The Nutrition Dashboard provides users with a centralized view of their personalized nutrition information. It brings together the generated diet chart, water-intake information, fitness goal, and other relevant user data.

The dashboard retrieves the required information through backend APIs and presents it using interactive React components. Users can view their current diet recommendations and food-analysis results from a single interface.

The module also provides a convenient way for users to monitor their nutrition-related activities and review their personalized information.

Module 8: API Integration & AI Communication

The API Integration module acts as the communication layer between the React frontend, Node.js/Express.js backend, database, and external AI services.

REST APIs are used to send user information from the frontend to the backend and retrieve generated diet plans, food-analysis results, and other nutrition-related information.

The backend communicates with AI APIs for personalized diet generation and food image analysis. The responses from these services are processed by the backend before being delivered to the React frontend.

This module provides a structured connection between the different components of Nutri AI and allows the application to integrate AI functionality into the MERN architecture.

FRONTEND CODE

```
import express from "express";
import path from "path";
import { createServer as createViteServer } from "vite";
import { GoogleGenAI, Type } from "@google/genai";
import dotenv from "dotenv";

dotenv.config();

const app = express();
const PORT = 3000;

app.use(express.json({ limit: '10mb' }));

const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY,
  httpOptions: {
    headers: {
      'User-Agent': 'aistudio-build',
    }
  }
});

app.post("/api/generate-diet", async (req, res) => {
  try {
    const {
      name, age, gender, height, weight,
      activityLevel, healthGoal,
      foodPreference, allergies, foodsToAvoid,
      mealsPerDay, preferredCuisine, budget, waterIntake,
      bmi, bmr, calorieRequirement,
      proteinRequirement, carbsRequirement, fatRequirement
    } = req.body;
```

```
const prompt = `
Generate a personalized 7-day diet plan for ${name}, a ${age} year old ${gender}.
You MUST provide exactly 7 days in the "days" array (dayNumber 1 through 7). Do not stop at Day 1. Each of the 7 days must have its own breakfast, morningSnack, lunch, eveningSnack, and dinner.
Height: ${height} cm, Weight: ${weight} kg.
Activity Level: ${activityLevel}.
Health Goal: ${healthGoal}.
Food Preference: ${foodPreference}.
Allergies: ${allergies || 'None'}.
Foods to avoid: ${foodsToAvoid || 'None'}.
Meals per day: ${mealsPerDay || 'Standard 3 meals + 2 snacks'}.
Preferred Cuisine: ${preferredCuisine || 'Any'}.
Budget: ${budget || 'Average'}.

Nutritional Targets:
Daily Calories: ${calorieRequirement} kcal
Protein: ${proteinRequirement} g
Carbs: ${carbsRequirement} g
Fat: ${fatRequirement} g

Ensure the plan meets these exact macro targets as closely as possible, and provide realistic food options.
Also ensure you cover micronutrients properly (Iron, Calcium, Vit A, Vit B1, Vit B2, Vit B3, Vit B6, Vit B12, Vit C, Vit D, Vit E, Vit K, Folate, Magnesium, Zinc, Potassium).
If the generated diet does not adequately cover a nutrient, you MUST provide suitable food sources to improve it in the micronutrientSuggestions array.
Do not suggest foods from their allergies or 'foods to avoid' list.
Provide realistic Indian food examples (rice, idli, dosa, dal, paneer, chicken, etc.) if Indian cuisine is preferred, else diverse healthy options.
`;

const mealSchema = {
```

```
  type: Type.OBJECT,
  properties: {
    foodName: { type: Type.STRING },
    portionSize: { type: Type.STRING },
    calories: { type: Type.NUMBER },
    protein: { type: Type.NUMBER },
    carbs: { type: Type.NUMBER },
    fat: { type: Type.NUMBER },
  },
  required: ["foodName", "portionSize", "calories", "protein", "carbs",
"fat"]
};

    const response = await ai.models.generateContent({
      model: "gemini-3.5-flash-lite",
      contents: prompt,
      config: {
        responseMimeType: "application/json",
        responseSchema: {
          type: Type.OBJECT,
          properties: {
            days: {
              type: Type.ARRAY,
              items: {
                type: Type.OBJECT,
                properties: {
                  dayNumber: { type: Type.INTEGER },
                  meals: {
                    type: Type.OBJECT,
                    properties: {
                      breakfast: mealSchema,
                      morningSnack: mealSchema,
                      lunch: mealSchema,
                      eveningSnack: mealSchema,
                      dinner: mealSchema,
```

```
            },
            required: ["breakfast", "morningSnack", "lunch",
"eveningSnack", "dinner"]
          },
          summary: {
            type: Type.OBJECT,
            properties: {
              calories: { type: Type.NUMBER },
              protein: { type: Type.NUMBER },
              carbs: { type: Type.NUMBER },
              fat: { type: Type.NUMBER },
              micronutrients: {
                type: Type.OBJECT,
                properties: {
                  ironCoveragePct: { type: Type.NUMBER },
                  calciumCoveragePct: { type: Type.NUMBER },
                  vitACoveragePct: { type: Type.NUMBER },
                  vitB12CoveragePct: { type: Type.NUMBER },
                  vitCCoveragePct: { type: Type.NUMBER },
                  vitDCoveragePct: { type: Type.NUMBER },
                  magnesiumCoveragePct: { type: Type.NUMBER },
                  zincCoveragePct: { type: Type.NUMBER },
                  potassiumCoveragePct: { type: Type.NUMBER },
                },
                required: ["ironCoveragePct", "calciumCoveragePct",
"vitACoveragePct", "vitB12CoveragePct", "vitCCoveragePct",
"vitDCoveragePct", "magnesiumCoveragePct", "zincCoveragePct",
"potassiumCoveragePct"]
              }
            },
            required: ["calories", "protein", "carbs", "fat",
"micronutrients"]
          }
        },
        required: ["dayNumber", "meals", "summary"]
      }
```

```
        },
        micronutrientSuggestions: {
          type: Type.ARRAY,
          items: {
            type: Type.OBJECT,
            properties: {
              nutrient: { type: Type.STRING },
              suggestion: { type: Type.STRING }
            }
          }
        }
      }
    }
  });

  const data = JSON.parse(response.text?.trim() || "{}");
  res.json(data);
 } catch (error) {
  console.error("Error generating diet:", error);
  res.status(500).json({ error: "Failed to generate diet" });
 }
});

app.post("/api/scan-food", async (req, res) => {
 try {
  const { image, targets, user } = req.body; // image is base64 data url

  if (!image) {
   return res.status(400).json({ error: "Image is required" });
  }

  const base64Data = image.split(',')[1];
  const mimeType = image.split(';')[0].split(':')[1];
```

BACKEND CODE

```
import { useState } from 'react';
import InputForm from './components/InputForm';
import Dashboard from './components/Dashboard';
import { UserProfile, NutritionTargets, DietPlan } from './types';
import { calculateNutrition } from './utils/nutrition';

export default function App() {
  const [userProfile, setUserProfile] = useState<UserProfile | null>(null);
  const [nutritionTargets, setNutritionTargets] = useState<NutritionTargets | null>(null);
  const [dietPlan, setDietPlan] = useState<DietPlan | null>(null);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const generateDiet = async (profile: UserProfile, targets: NutritionTargets) => {
    setIsLoading(true);
    setError(null);
    try {
      const response = await fetch('/api/generate-diet', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          ...profile,
          ...targets
        })
      });

      if (!response.ok) {
        throw new Error('Failed to generate diet plan');
      }

      const data = await response.json();
      setDietPlan(data);
      setUserProfile(profile);
      setNutritionTargets(targets);
    } catch (err: any) {
      console.error(err);
      setError(err.message || 'An error occurred while generating the diet plan.');
    } finally {
      setIsLoading(false);
    }
  };

  const handleFormSubmit = (data: UserProfile) => {
    const targets = calculateNutrition(data);
    generateDiet(data, targets);
  };
```

```
const handleRegenerate = () => {
    if (userProfile && nutritionTargets) {
      generateDiet(userProfile, nutritionTargets);
    }
  };

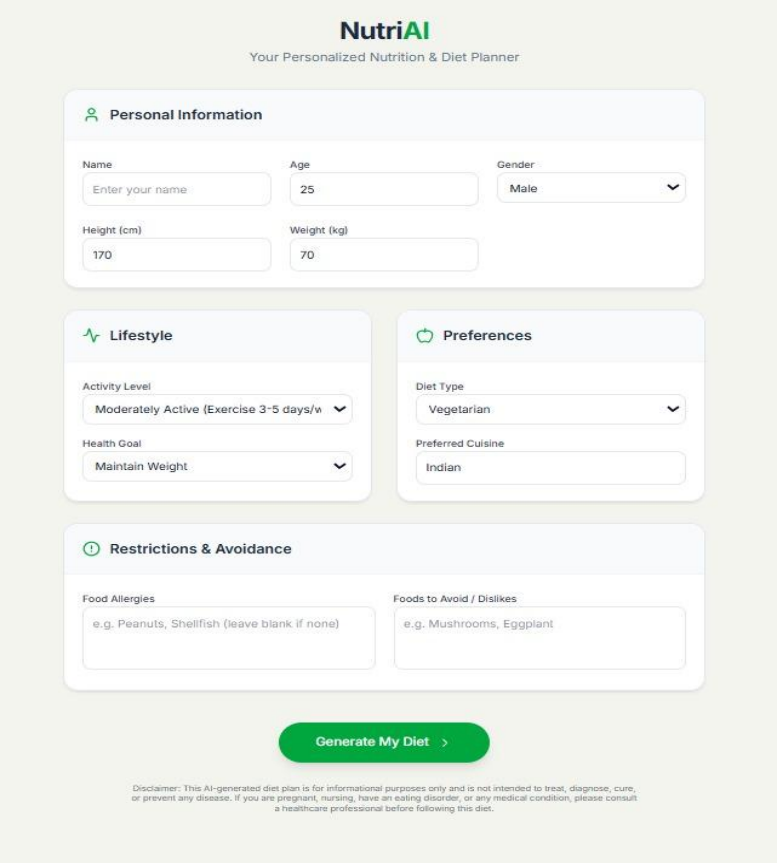
  const handleBack = () => {
    setDietPlan(null);
  };

  return (
    <div className="min-h-screen bg-organic font-sans text-slate-900">
      <main className="py-8 px-4 h-full flex flex-col">
        {error && (
          <div className="max-w-4xl mx-auto mb-6 p-4 bg-red-50 border border-red-200 text-
red-700 rounded-lg">
            {error}
          </div>
        )}

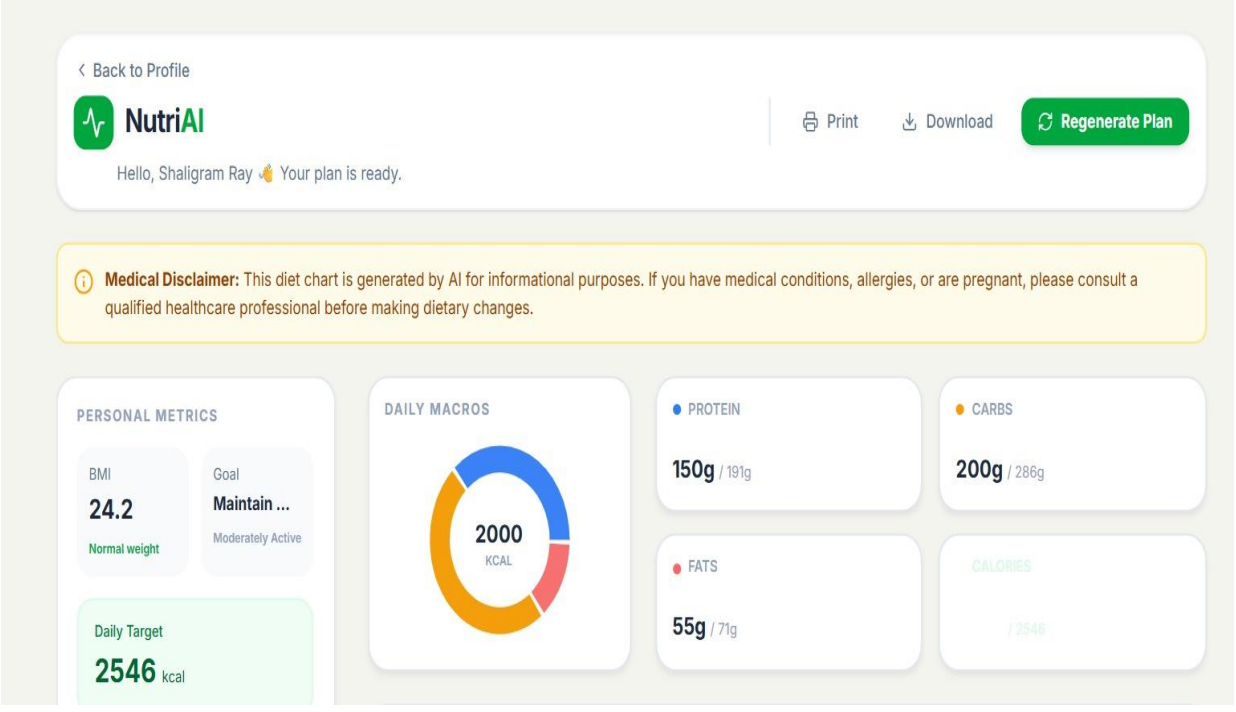
        {!dietPlan ? (
          <InputForm onSubmit={handleFormSubmit} isLoading={isLoading} />
        ) : (
          <Dashboard
            user={userProfile!}
            targets={nutritionTargets!}
            dietPlan={dietPlan}
            onRegenerate={handleRegenerate}
            onBack={handleBack}
            isRegenerating={isLoading}
          />
        )}
      </main>
    </div>
  );
}
```

OUTPUT

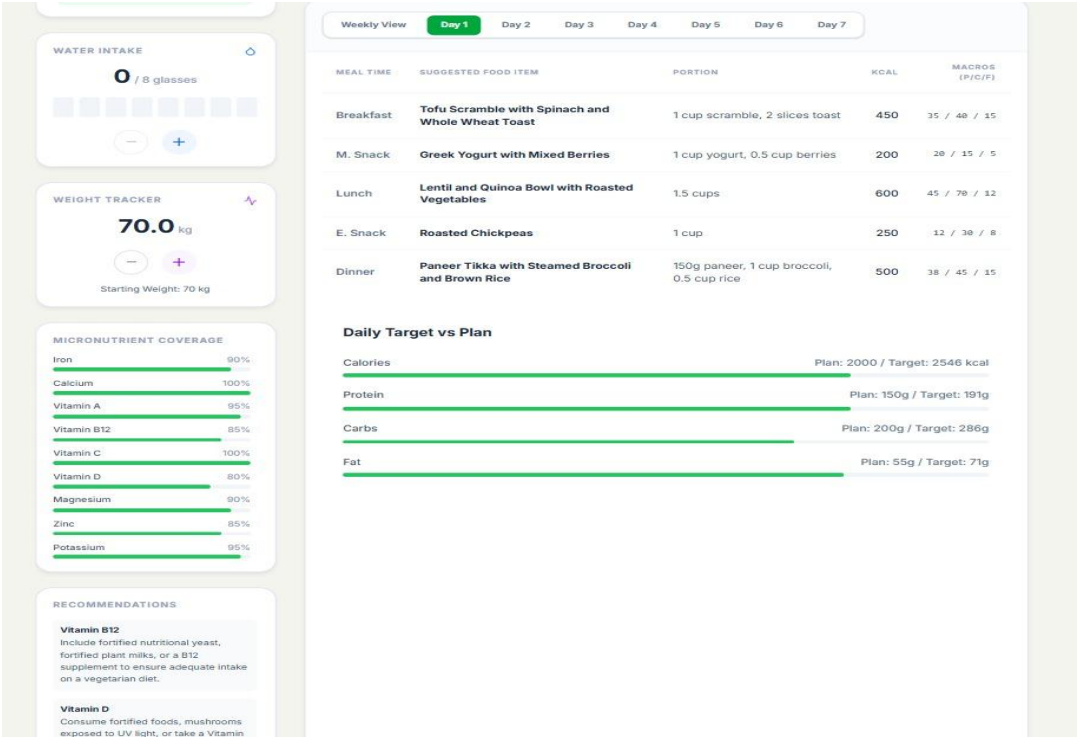
Home Page:



Dashboard:



Daily Tracker Page:



Discover Nutrients:

